

OREN-X: Octree Residual Network for Real-Time Multi-Modal Mapping

Zhirui Dai¹ Qihao Qian¹ Dinh Minh Nguyen² Quan-Dung Pham² Kiana Bronder³
Carlos Nieto-Granda⁴ Yiyu Chen² Quan Nguyen⁵ Nikolay Atanasov¹

Abstract—To achieve general-purpose autonomy over long horizons, a robot needs to maintain spatial environment information that supports a variety of tasks: geometry for planning and control, radiance for rendering and relocalization, and vision-language features for open-vocabulary grounding. Existing methods represent and estimate each modality separately, multiplying memory and compute cost while forgoing potential synergy among the representations. We develop OREN-X, an online mapping method that uses an octree in 3D space as a shared data structure for indexing and storing a multi-modal field, capturing geometric, radiance, and vision-language information. OREN-X provides efficient unified storage and retrieval of these data in explicit/implicit and full/compressed form. Our unified representation yields cross-modality synergy: SDF estimates are sharpened by occupancy and radiance, while GPU-based ray-octree traversal and octree query enable real-time rendering. We also use online dictionary learning to compress the vision-language features, shrinking them 3.7× below full per-vertex storage while raising the query accuracy. On Replica, OREN-X maps in real time (80+ fps for SDF and 30+ fps for all four modalities), improves near-surface SDF accuracy by 33% over single-modality baselines, and improves mean open-vocabulary 3D mIoU by 71% and mean accuracy by 61% over the best prior method.

I. INTRODUCTION

Long-horizon general-purpose autonomy requires a robot to maintain multiple types of information about its environment. Modern robotics stacks often maintain a separate representation per modality, such as a geometric field (distance or occupancy) for planning and control, a radiance field for rendering and relocalization, and a vision-language field for open-vocabulary querying and instruction grounding. This is inefficient in two ways. First, it multiplies memory and computation with the number of representations, a critical bottleneck for real-time, long-horizon operation. Second, independent modalities cannot exploit synergies among them.

Many existing methods target a single modality. Online SDF mapping fuses truncated SDF into discrete grids [1], [2]

¹The authors are with the Department of Electrical and Computer Engineering, University of California San Diego, La Jolla, CA 92093, USA, e-mails: {zhundai, q2qian, natanasov}@ucsd.edu.

²Dinh Minh Nguyen, Quan-Dung Pham and Yiyu Chen are with VinMotion, e-mails: v.minhnd59@vinmotion.net, pqdung@snu.ac.kr, yiyuc@usc.edu.

³Kiana Bronder is with Parsons, e-mail: kiana.bronder@parsons.us.

⁴Carlos Nieto-Granda is with the U.S. DEVCOM Army Research Laboratory, Adelphi, MD 20783, USA, e-mail: carlos.p.nieto2.civ@army.mil.

⁵Quan Nguyen is with the University of Southern California, Los Angeles, CA 90089, USA, e-mail: quann@usc.edu.

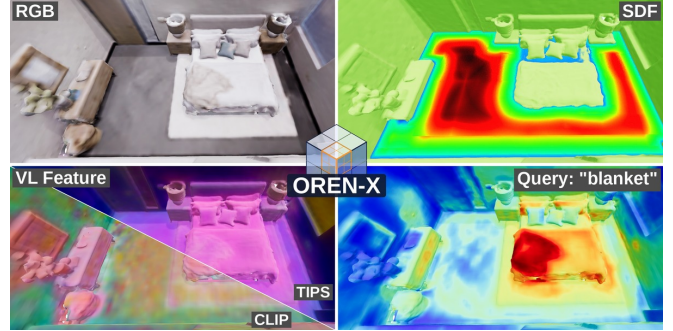


Fig. 1: **OREN-X at a glance.** From RGB-D data, OREN-X builds a single octree-based multi-modal field that jointly represents geometry, radiance and vision-language features (shown here for two backbones, CLIP and TIPS), supporting real-time open-vocabulary queries (e.g. “blanket”) localized directly in 3D.

or fits continuous neural fields [3]–[6]. NeRF- and 3DGS-based methods learn radiance [7], [8]. A growing body of work learns vision-language (VL) fields [9]–[11]. A robot that needs all of these capabilities must run several pipelines side by side. Storing multiple modalities in one spatial structure is not enough to unify them. Unless the modalities also share how their parameters are stored, queried, and trained, each keeps its own parameters and updates, benefiting from none of the others. A unified representation therefore needs three properties: common storage across modalities, one indexing scheme that queries them together, and joint training in which each modality improves the others.

We present OREN-X, an online multi-modal mapping framework that represents all modalities with a single octree. Every modality stores its parameters, as explicit values, latent features decoded by a small network, or both, at the octree vertices, in full or compressed form to save memory. All modalities use the same octree lookup to locate and interpolate their parameters to obtain continuous values at any query point, and are optimized together online with per-modality and cross-modality losses that let one modality improve another. We demonstrate OREN-X with four modalities: signed distance, occupancy, radiance and vision-language.

Joint training yields cross-modality synergy. Radiance supervision through an SDF-derived density improves SDF accuracy, and an SDF-occupancy consistency loss improves robustness under strong depth noise. The shared octree makes the VL features queryable in 3D by interpolation and in 2D by rendering with radiance. The octree also reduces computation because it supports both GPU ray-octree traversal for rendering and fast tile-based rasterization. Because the VL features need the most memory, we develop an online

dictionary learning method that compresses them, improves query accuracy, and scores text queries directly in low-dimensional code space. Our contributions are as follows.

- **Unified multi-modal mapping.** We develop an octree that provides common storage, shared indexing, and joint training for different modalities, such as signed distance, occupancy, radiance, and vision-language.
- **Cross-modality synergy.** Radiance supervision improves SDF accuracy, an SDF-occupancy consistency loss improves noise robustness, and shared indexing enables both 2D and 3D open-vocabulary queries.
- **Memory and compute efficiency.** Shared indexing lets one octree traversal serve all modalities in real time. We develop an online dictionary learning method to compress the VL features with higher query accuracy and to answer text queries directly in code space.

II. RELATED WORK

Online geometric mapping. Dense geometric mapping has long relied on volumetric fusion of depth into discrete grids, from early truncated-SDF fusion [12] made real-time on a GPU by KinectFusion [13], to probabilistic occupancy octrees that bound memory and distinguish free from unknown space for planning [14], to incremental Euclidean SDF extraction from the TSDF for collision checking on CPU [1] and GPU [2]. These grids, however, are limited by voxel resolution. Occupancy Networks instead represent geometry as a continuous neural decision boundary [15]. Recent methods fit continuous, neural or hybrid SDF fields online [3], [4], often on hierarchical feature grids that concentrate parameters near surfaces [7], [16], [17]. An octree suits such sparse 3D data, offering fast lookup, multi-resolution detail, and incremental growth [16], [18]. OREN [6] achieves online SDF reconstruction using an explicit coarse prior and an implicit neural residual. A single geometric field, however, does not provide semantic information a robot needs to execute complex tasks autonomously.

Radiance fields and rendering. Neural radiance fields (NeRF) [19] and their accelerated grid variants [17], [20] render appearance by volumetric ray marching, while surface-oriented formulations tie appearance to geometry through an SDF-based density term [21]. 3D Gaussian Splatting (3DGS) [8] and its surface-accurate variants [22] instead render appearance by fast tile-based rasterization, and sparse-voxel rasterizers reach comparable quality with voxel primitives [18]. These two families, ray-based and tile-based rasterization, trade query flexibility against speed, and our method supports both on one octree, where GPU-based ray-octree traversal and query keep the ray-based rasterizer efficient for real-time radiance optimization while the tile-based rasterizer renders images faster.

Open-vocabulary language fields. Semantic octrees [23] and 3D scene graphs such as Hydra [24] attach semantic labels to geometry but their label sets are fixed before deployment. Vision-language (VL) backbones such as CLIP [25], DINOv3 [26] with text alignment [27], [28], and TIPsv2 [29] remove this restriction by embedding images and text

in a shared feature space, but since these features live in the image plane, they must be fused into a spatial representation to become consistent across views and queryable in 3D. LERF distills VL features into a NeRF [9], 3DGS-based methods store them in each Gaussian [10], [30], [31], and point- or graph-based maps attach them to points or objects [32]–[34], usually compressing the memory-prohibitive per-primitive features by autoencoders [10], [35], quantization [30], [31], or online dictionary learning [11], [36]. These methods, however, layer the language field on an appearance reconstruction or keep it as a separate semantic map rather than maintaining it with the geometry a robot needs for navigation. In contrast, OREN-X maintains VL features jointly with geometry and appearance in one octree, and improves memory and open-vocabulary retrieval using online dictionary learning of a compressed VL feature code.

III. PROBLEM STATEMENT

We aim to reconstruct a single multi-modal field $f = (f_c)_{c \in \mathcal{M}}$ with four modalities, $\mathcal{M} = \{\text{sdf}, \text{occ}, \text{rad}, \text{vl}\}$: signed distance (sdf), occupancy (occ), radiance (rad) and vision-language features (vl). The component $f_c : \mathcal{X}_c \rightarrow \mathbb{R}^{d_c}$ of modality c has domain $\mathcal{X}_c = \mathbb{R}^3$ for sdf, occ and vl, and $\mathcal{X}_{\text{rad}} = \mathbb{R}^3 \times \mathbb{S}^2$, which adds the viewing direction.

At time t , a robot receives RGB image $\mathbf{I}_t$, depth image $\mathbf{D}_t$ and camera pose $\xi_t \in SE(3)$, and a VL model computes pixel-aligned features Φ_t . Back-projecting pixels with depth and pose gives training data $\mathcal{D}_{c,t} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_i$ per modality c , where $\mathbf{x}_i$ is a 3D position or camera ray and $\mathbf{y}_i \in \mathbb{R}^{d_c}$ is its label. The RGB image provides pixel colors for rad, and the depth image provides observed surface points for sdf and occ. For sdf and occ, we sample points along each ray, in free space before the surface point and around it. The sdf label of a sample is its distance to the nearest observed surface, negated behind it, and the occ label is 0 in free space, 0.75 at the surface (logit ≈ 1.0), and 1 behind it.

Problem. Given the stream of RGB images $\mathbf{I}_\tau$, depth images $\mathbf{D}_\tau$, camera poses ξ_τ and vision-language features Φ_τ for $\tau \leq t$, estimate online the multi-modal field f that minimizes

$$\sum_j \lambda_j \mathcal{L}_j \left(\{f_c\}_{c \in \mathcal{S}_j}; \mathcal{D}_t \right), \quad (1)$$

with bounded computation and memory, where $\mathcal{D}_t = \bigcup_{c, \tau \leq t} \mathcal{D}_{c,\tau}$, and each loss term $\mathcal{L}_j$ with weight λ_j depends on a subset of modalities $\mathcal{S}_j \subseteq \mathcal{M}$ and evaluates them at positions or integrates them along rays. A term with $|\mathcal{S}_j| > 1$ is a cross-modality loss that couples the modalities in $\mathcal{S}_j$.

IV. TECHNICAL APPROACH

OREN-X performs online multi-modal mapping: from a posed RGB-D stream, it estimates all four modalities of one field in real time with a single octree. Sec. IV-A develops an octree representation of the multi-modal field and its online update. Sec. IV-B describes how each modality is decoded and supervised, including the cross-modality loss terms. Sec. IV-C presents our online dictionary learning algorithm that compresses the VL features.

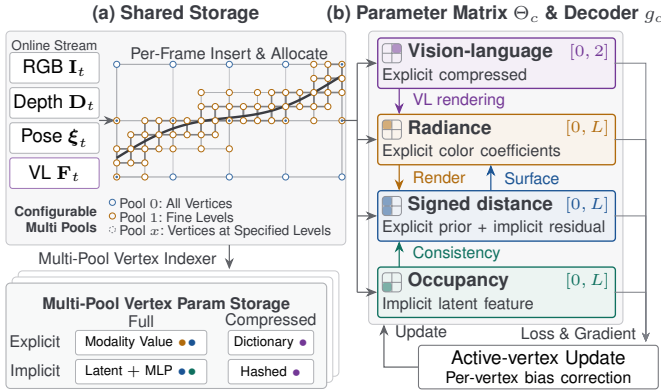


Fig. 2: **OREN-X overview.** (a) A stream of RGB-D and VL features updates a multi-modal field with four modalities (sdf, occ, rad, and vl). Observed points allocate octree voxels at different levels, and multi-pool indexing maps a query point to its surrounding vertices’ rows in each modality’s parameter matrix Θ_c . (b) Each modality’s parameters are explicit values, implicit features, or both, stored in full or compressed form, decoded by the decoder g_c to produce modality values and loss terms. The loss terms are summed and optimized by a single backward pass, updating only active vertices with one Adam step using per-vertex bias correction.

A. Unified Multi-Modal Mapping

Except for learnable weights $\mathbf{W}$ shared across vertices by the decoder g_c , OREN-X stores all modalities’ parameters at the octree vertices, organized by three designs. First, multi-pool indexing lets each modality share the octree at its own spatial resolution. Second, all modalities share one per-vertex parameterization, where a vertex stores explicit values, implicit features, or both, in full or compressed form, so each modality uses the combination that suits it. Third, an active-vertex online update keeps the per-frame cost bounded.

Octree parameterization. OREN-X represents the field f on an octree $\mathcal{O}$, which partitions space into cubic voxels at levels $\ell = 0, \dots, L$, where level 0 is the finest with size r_0 and level ℓ has size $r_\ell = r_0 2^\ell$. Each allocated voxel’s parent is also allocated, so voxels form a tree, with each voxel’s eight corners as its vertices. Each modality $c \in \mathcal{M}$ has a parameter matrix $\Theta_c \in \mathbb{R}^{V_c \times n_c}$, whose row $\Theta_c[k]$ is the parameter vector of the k -th vertex allocated to c . An indexing function $\mathcal{P}_c$ maps a query position $\mathbf{x}$ to the vertices around it, and a decoder g_c , with weights $\mathbf{W}$, predicts modality c ’s value,

$$f_c(\mathbf{u}) = g_c\left(\mathbf{u}, \sum_{k \in \mathcal{P}_c(\mathbf{x})} w_k(\mathbf{x}) \Theta_c[k]\right), \quad (2)$$

where $\mathbf{u} \in \mathcal{X}_c$ is the input, $\mathbf{x}$ is its position part, and $w_k(\mathbf{x})$ are interpolation weights, e.g., trilinear. OREN-X estimates $\{\Theta_c\}_{c \in \mathcal{M}}$ and $\mathbf{W}$ online to optimize (1), as shown in Fig. 2. **Multi-pool indexing.** OREN-X gives each modality $c \in \mathcal{M}$ its own vertex pool spanning a contiguous level range $\mathcal{L}_c = \{\ell : \ell_c^{\text{lb}} \leq \ell \leq \ell_c^{\text{ub}}\}$, so heterogeneous modalities share one octree at different spatial resolutions and extents. Each vertex $\mathbf{v}$ is keyed by its Morton code [18],

$$\mathcal{T}(\mathbf{v}, r_0) = \text{Morton}(\lfloor \mathbf{v}/r_0 \rfloor), \quad (3)$$

which interleaves the bits of the three integer coordinates $\lfloor \mathbf{v}/r_0 \rfloor$ into a single, level-independent integer. Using this

key, the indexing function $\mathcal{P}_c$ gathers, for every $\ell \in \mathcal{L}_c$, the eight corners $\mathcal{C}_\ell(\mathbf{x})$ of the level- ℓ voxel containing $\mathbf{x}$,

$$\mathcal{P}_c(\mathbf{x}) = \{(\ell, \mathcal{T}(\mathbf{v}, r_0)) : \ell \in \mathcal{L}_c, \mathbf{v} \in \mathcal{C}_\ell(\mathbf{x})\}. \quad (4)$$

This multi-pool design allows each modality to pick its own level range and allocate parameters only where needed, while still sharing the same octree for efficient indexing and update.

Per-vertex representation. Each row $\Theta_c[k]$ can contain explicit values and implicit features. The explicit part stores the modality’s value or spatial derivatives at the vertex, such as an SDF value with its gradient or a color, so interpolating it directly estimates f_c . The implicit part stores a latent feature that g_c decodes into the modality value or a residual correcting the explicit part’s estimate of f_c . Either part can instead be stored in compressed form, as a shorter vector of weights over a shared fixed-capacity dictionary, from which the full vector is recovered as a weighted sum of dictionary atoms. Fig. 2 shows each modality’s configuration, chosen by role and empirical experience. SDF keeps OREN’s explicit prior [6] for coarse geometry and adds an implicit residual for detail. Occupancy decodes the implicit feature with its own MLP. Radiance stores colors explicitly for direct rendering. VL features are stored explicitly for linear decoding, so text queries score directly in the code space (Sec. IV-C), and are compressed by our online dictionary learning algorithm. This parameterization admits other combinations of explicit, implicit, and compressed representations, but ablating them is beyond the scope of this paper.

Online update. Following OREN [6], our formulation grows the octree with each new frame and maintains a key-frame set to sample training frames alongside the current one. Streaming, partially observed supervision updates each vertex a different number of times, since vertex k gets a gradient only on frames where it is active, so its effective step count t_k varies across vertices and departs from the global iteration counter. Vanilla Adam, whose bias correction assumes every parameter is stepped every iteration, over-shrinks moment estimates for rarely observed vertices. We instead keep a per-vertex step count t_k , incremented only when k is active, to bias-correct the first- and second-moment estimates:

$$m_k \leftarrow \beta_1 m_k + (1 - \beta_1) g_k, \quad v_k \leftarrow \beta_2 v_k + (1 - \beta_2) g_k^2, \\ \Theta_c[k] \leftarrow \Theta_c[k] - \alpha \frac{m_k / (1 - \beta_1^{t_k})}{\sqrt{v_k / (1 - \beta_2^{t_k}) + \epsilon}}, \quad (5)$$

where $\alpha, \beta_1, \beta_2, \epsilon$ are the usual Adam constants and g_k is the accumulated gradient for $\Theta_c[k]$. This per-vertex update is also sparse, since only active vertices carry non-zero gradients, so the moment and parameter updates touch those vertices and per-frame cost scales with the observed region rather than the global map. The active set is the union of all modalities’ active vertices, recorded by octree queries.

B. Cross-Modality Synergy

Co-locating modalities on one octree lets them reinforce one another in three ways: (i) *quality*: modalities supervise

one another, (ii) *compute*: one top-down octree descent, ray-octree traversal, or in-frustum voxel filtering serves several modalities at once, (iii) *memory*: the shared allocation, indexing and updates are paid once, rather than $|\mathcal{M}|$ times, across modalities. We describe each modality below and emphasize its coupling to the multi-modal representation.

Signed distance. We adopt OREN’s SDF representation and loss terms [6]. Each vertex k with position $\mathbf{x}_k \in \mathbb{R}^3$ stores an SDF value $d_k \in \mathbb{R}$ and gradient $\mathbf{g}_k \in \mathbb{R}^3$ as the explicit part, and a feature $\mathbf{f}_k \in \mathbb{R}^{16}$ as the implicit part, so $\Theta_{\text{sdf}}[k] = (d_k, \mathbf{g}_k, \mathbf{f}_k)$. Following (2), the decoder g_{sdf} extrapolates the explicit part of the finest voxel’s corner $k \in \mathcal{P}_{\text{sdf}}(\mathbf{x})$ to the query point, $d_k(\mathbf{x}) = d_k + \mathbf{g}_k^\top (\mathbf{x} - \mathbf{x}_k)$, and interpolates the results with the trilinear weights $w_k(\mathbf{x})$ into a coarse estimate $d_{\text{ga}}(\mathbf{x})$. A small MLP D with weights $\mathbf{W}$ then adds a residual from $d_{\text{ga}}(\mathbf{x})$ and the interpolated feature $\mathbf{f} = \sum_k w_k(\mathbf{x}) \mathbf{f}_k$, $f_{\text{sdf}}(\mathbf{x}) = d_{\text{ga}}(\mathbf{x}) + D(d_{\text{ga}}(\mathbf{x}), \mathbf{f}; \mathbf{W})$. Since a valid SDF has unit gradient norm, we regularize the gradient with the Eikonal loss, $||\nabla f_{\text{sdf}}(\mathbf{x})|| - 1$, computed by finite differences. To speed it up, a CUDA kernel looks up the sample point’s containing voxel and its six stencil points simultaneously, since paired stencil points are usually in the same voxel. The kernel executes an extra descent only when a stencil point falls into a neighboring voxel, so we reuse shared indexing without an extra octree traversal (*compute*).

Occupancy. The occupancy at a point $\mathbf{x}$ is predicted as a logit $\hat{\ell}(\mathbf{x})$, decoded by an MLP from its own implicit feature, and supervised by the occupancy label $y(\mathbf{x})$ with a binary cross-entropy (BCE) loss. A sign-consistency term then lets the detached occupancy teach the SDF its sign, penalizing SDF predictions $\hat{s}(\mathbf{x})$ that fall on the wrong side of a confident occupancy logit,

$$\begin{aligned} \mathcal{L}_{\text{occ}} &= \text{BCE}(\hat{\ell}(\mathbf{x}), y(\mathbf{x})), \\ \mathcal{L}_{\text{cons}} &= \max(0, \hat{s}(\mathbf{x}) \text{sg}[\text{sign} \hat{\ell}(\mathbf{x})]), \end{aligned} \quad (6)$$

with $\mathcal{L}_{\text{cons}}$ restricted to confident points $|\hat{\ell}(\mathbf{x})| > \eta$, and $\text{sg}[\cdot]$ the stop-gradient. Because $\hat{\ell}$ is detached, occupancy makes the SDF sign robust under strong depth noise without the SDF pulling occupancy back (*quality*).

Radiance. The radiance modality stores an RGB color at each vertex, so $n_{\text{rad}} = 3$, supervised by images. Each pixel defines a ray $\mathbf{r}$ through it from the camera center, and we predict its color $\hat{C}(\mathbf{r})$ from the field. Following volume rendering [19], we evaluate the field at K points along the ray from the camera to the surface, and composite their colors,

$$\hat{C}(\mathbf{r}) = \sum_{k=1}^K T_k \alpha_k \mathbf{c}_k, \quad T_k = \prod_{j < k} (1 - \alpha_j), \quad (7)$$

where $\mathbf{c}_k$ is the color decoded at sample k , $\alpha_k = 1 - e^{-\sigma_k \delta_k}$ is the probability the ray stops between sample k and the next one at distance δ_k , and T_k is the probability the ray reaches sample k . Instead of learning a separate density σ_k , we derive it from the SDF [21], so the photometric loss on $\hat{C}(\mathbf{r})$ couples the SDF and radiance modalities. The SDF-induced surface location lets the renderer sample on the surface more accurately, and photometric gradients flow back through the

Algorithm 1 Online dictionary: accumulate, initialize, grow.

Require: decay ρ , energy threshold ϵ_{dict} , capacity G_{max}

- 1: $D \leftarrow []$, $R \leftarrow \mathbf{0}$, $E \leftarrow 0$ $\triangleright$ dictionary, residual scatter, energy
- 2: **procedure** ACCUMULATE(V) $\triangleright V \in \mathbb{R}^{M \times d}$, one frame
- 3: $A \leftarrow VD$; $R_\perp \leftarrow V - AD^\top$
 $\triangleright$ projected codes and orthogonal-complement residual
- 4: $R \leftarrow \rho R + R_\perp R_\perp^\top$; $E \leftarrow \rho E + \|V\|_F^2$
- 5: **procedure** BOOTSTRAP $\triangleright$ one-shot initialization
- 6: $(\Lambda, U) \leftarrow \text{EIGH}(R)$
- 7: $D \leftarrow [u_j : \lambda_j/E > \epsilon_{\text{dict}}]$ $\triangleright$ keep dominant directions
- 8: $R \leftarrow R - \sum_j \lambda_j u_j u_j^\top$ $\triangleright$ deflate absorbed energy
- 9: **procedure** GROW $\triangleright$ append-only
- 10: **while** $|D| < G_{\text{max}}$ **do**
- 11: $(\lambda, u) \leftarrow \text{POWERITER}(R, D)$
- 12: **if** $u = \emptyset$ **or** $\lambda/E \leq \epsilon_{\text{dict}}$ **then**
- 13: **break**
- 14: $D \leftarrow [D, u]$; $R \leftarrow R - \lambda u u^\top$
- 15: **function** POWERITER(C, D)
- 16: $\text{PROJ}(x) \triangleq x - DD^\top x$
- 17: **if** $|D| \geq d$ **then** $\triangleright$ complement empty
- 18: **return** $(0, \emptyset)$
- 19: $u \leftarrow \text{PROJ}(x)$ for a random unit x ; $u \leftarrow u/\|u\|$
- 20: **repeat**
- 21: $v \leftarrow \text{PROJ}(Cu)$; $v \leftarrow v/\|v\|$
- 22: $\Delta \leftarrow \|v - u\|$; $u \leftarrow v$
- 23: **until** $\Delta < \delta$ **or** max iterations
- 24: **return** $(u^\top Cu, u)$ $\triangleright$ Rayleigh quotient, unit eigenvector

density term to refine the SDF, so SDF and radiance improve each other (*quality*). We adopt the tile-based rasterizer of [18] for real-time octree-based rendering, but since it only touches in-frustum voxels per frame and converges slowly during training, we add a ray-based rasterizer, whose GPU kernel implements the parametric ray-octree traversal [37], that trains more efficiently and converges faster. We thus train with the ray-based rasterizer and render with the tile-based one on the same octree (*compute*).

Vision-language. We store the VL features in a near-surface band of the octree, co-located with the radiance modality. So, OREN-X supports both rendering the color and the VL features within the same 2D rendering pass, as well as querying the VL features in 3D space (*compute*). It is prohibitive to store the VL features in full dimension for every vertex, so we compress them into a sparse code over a shared dictionary (*memory*), detailed in Sec. IV-C.

C. Memory Compression for Vision-Language Features

Unlike most existing methods that require pre-trained dictionaries [30], [31], OREN-X learns the dictionary online from the streaming features.

Online dictionary. We use an append-only rule to grow the basis elements (atoms) of a dictionary. When current atoms leave enough energy of new observations unexplained, a new atom is added and never modified. The dictionary resembles online PCA [38] for VL features, except its atom count grows by an energy criterion and the atoms do not rotate with the subspace. Each vertex stores a vector of weights (code) over the atoms, instead of the raw feature. As a vertex is re-observed and the dictionary grows, its code is updated with the atoms fixed. Alg. 1 summarizes the procedure.

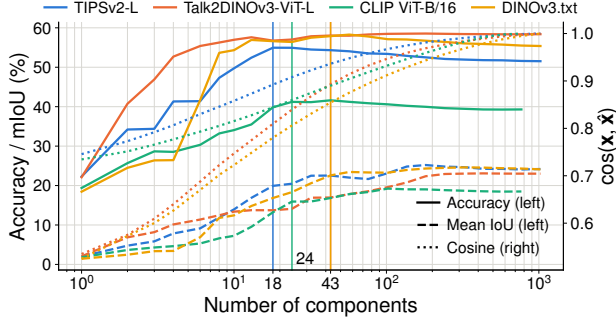


Fig. 3: **VL reconstruction vs. query accuracy.** As the dictionary size grows, open-vocabulary query accuracy saturates well before reconstruction fidelity does.

Text query. A text query is embedded by a text encoder as ϕ_q and scored against a VL feature $\mathbf{f}$ by cosine similarity,

$$\text{sim}(\mathbf{f}, \phi_q) = \frac{\mathbf{f}^\top \phi_q}{\|\mathbf{f}\| \|\phi_q\|}. \quad (8)$$

Following [9], a relevancy score contrasts the query with a set of negative queries $\{\phi_n^i\}$ and keeps the least favorable,

$$\text{rel}(\mathbf{f}, \phi_q) = \min_i \frac{e^{\text{sim}(\mathbf{f}, \phi_q)/\tau}}{e^{\text{sim}(\mathbf{f}, \phi_q)/\tau} + e^{\text{sim}(\mathbf{f}, \phi_n^i)/\tau}}, \quad (9)$$

where τ scales the similarities. Because decoding is linear, a text query is embedded once into the code space $\mathbb{R}^{|D|}$ and scored directly against per-vertex codes in the dictionary’s coordinates, without decoding back to the d_{v1} feature.

Compression analysis. Open-vocabulary querying scores features by cosine similarity (8) and relevancy (9), which depend on the few directions that separate concepts, not on faithful reconstruction. As shown in Fig. 3, the reconstruction fidelity is affected by low-variance directions that leave query-score ranking unchanged. As the dictionary grows, query accuracy saturates while reconstruction fidelity keeps improving, so the dictionary should only admit directions up to the saturation point. As shown in Fig. 4a, VL features typically concentrate in a cone around their mean rather than spreading isotropically, so a few directions capture most of their energy. The cones of most class pairs overlap (Fig. 4b), so a single feature is often ambiguous between classes, and aggregating features within a voxel raises query accuracy above the per-point level (Fig. 4d). Although feature spread within a voxel grows with voxel size (Fig. 4c), a per-voxel PCA with only $k = 18$ components already saturates query accuracy (Fig. 4d), so the features are locally low-rank and a compact code per vertex suffices.

V. EVALUATION

We evaluate whether OREN-X’s joint training of multiple modalities in one octree improves geometry (Sec. V-C) and radiance (Sec. V-D). We also compare the compressed VL features to baselines in 3D and 2D queries (Sec. V-E) and ablate the design choices in OREN-X (Sec. V-F).

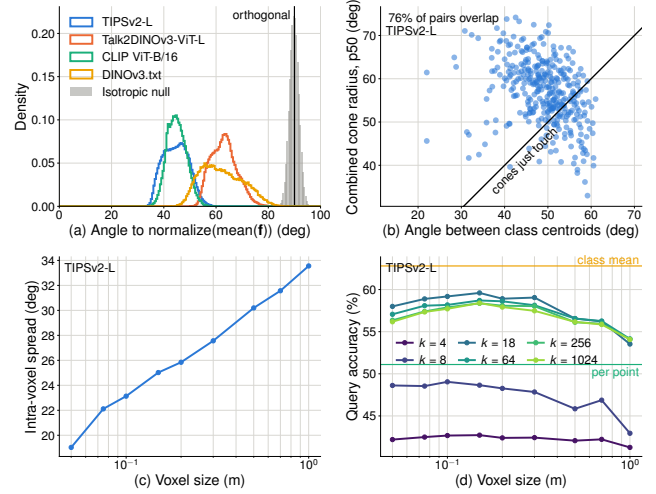


Fig. 4: **Geometry of VL features in Replica [39] room0.**

(a) Histogram of angles between each feature and the normalized mean feature. Features lie in a $35\text{--}80^\circ$ cone around the mean, far from the 90° expected for isotropic features. (b) Combined median cone radius vs. the angle between class centroids, per class pair. 76% of pairs overlap (above the diagonal), so a single feature is often ambiguous between classes. (c) Feature angular spread within a voxel grows with voxel size. (d) Query accuracy of per-voxel PCA features (k components) vs. voxel size, between per-point and class-mean references. Accuracy saturates by $k=18$ and peaks at intermediate voxel sizes.

A. Experiment Setup

Datasets. We evaluate on Replica (8 scenes) [39], which provides semantic ground truth for open-vocabulary evaluation, and TUM (3 sequences) [40], for which we generate pseudo-ground-truth labels using LatentAM’s [11] 10 classes and SAM 3 [41].

Baselines. For geometry, we compare with nvblox [2] (TSDF fusion) and OREN [6]. For radiance and VL, we compare with the offline LangSplatV2 [30] and two online methods, OnlineLangSplat (OLS) [35] and LatentAM [11], running OREN-X with a matched backbone for each since the baselines’ backbones differ, CLIP [25] for LangSplatV2 and OLS, and TIPSv2-L [29] for LatentAM. Absolute scores vary with the backbone, but OREN-X’s advantage over its matched baseline holds consistently across both. All methods use the ground-truth camera poses. Since the baselines are designed for 2D queries only, for 3D queries we inverse-distance-weight the features within 15 cm of the query point.

Metrics. For geometry, we report mesh accuracy, completion, Chamfer distance, and F-score at 5 cm, plus the mean absolute error (MAE) of the SDF and its gradient near ($|s| \leq 10$ cm) and far from the surface. For appearance, we report PSNR, SSIM, LPIPS, and depth L1. For VL, we report mIoU, mean accuracy (mAcc), and mAP in 3D and 2D, plus feature cosine similarity for reconstruction fidelity. We measure efficiency by FPS and peak allocated GPU memory on an NVIDIA RTX 5090.

Implementation. Unless stated otherwise, we use the same hyperparameters for all scenes. We use an 8-level octree with $r_0 = 0.1$ m, and encode radiance as a per-vertex $\mathbb{R}^3$ vector with sigmoid activation. The VL dictionary caps at

TABLE I: **Geometry** on Replica (8-scene mean). Mesh distances at a 5 cm threshold, matching OREN’s protocol. Best in **bold** and shaded green, second yellow.

Metric	nvblox	OREN [6]	OREN-X (ours)
Accuracy [cm] ↓	1.00	2.50	2.49
Completion [cm] ↓	2.17	2.12	2.14
Chamfer [cm] ↓	1.58	2.31	2.31
F-score [%] ↑	93.27	90.73	90.68
SDF MAE, All [cm] ↓	4.44	2.25	1.92
SDF MAE, Near [cm] ↓	5.47	1.70	1.14
SDF MAE, Far [cm] ↓	3.73	2.58	2.62
Grad. MAE, All [rad] ↓	0.272	0.160	0.147
Grad. MAE, Near [rad] ↓	0.369	0.122	0.101
Grad. MAE, Far [rad] ↓	0.222	0.176	0.176

$G_{\max} = 256$ atoms, with growth controlled by energy threshold $\epsilon_{\text{dict}} = 0.002$ and decay $\rho = 0.9$.

B. Qualitative Results

Fig. 1 shows OREN-X’s modalities on a Replica scene, and Fig. 5 compares our method with the radiance and VL baselines. In Fig. 5, OREN-X renders clean images, slightly smoother than LangSplatV2 and OLS, while LatentAM shows speckle artifacts. For open-vocabulary queries, the baselines produce noisy, diffuse, or missing responses, whereas OREN-X responds strongly, densely, and smoothly on the queried objects (blanket, table, monitor).

C. Geometry

In Table I, nvblox has the best mesh, since TSDF fusion reproduces the visible surface directly, but its truncated field has the largest SDF and gradient errors. Compared with OREN, OREN-X reduces the near-surface SDF error by 33% and the gradient error by 17%, with on-par far-field and mesh metrics while maintaining three more modalities, since occupancy and radiance supervise the SDF near the surface.

D. Appearance

In Table II, LangSplatV2 has the best images on both datasets but the worst depth, since its offline-optimized Gaussians are not tied to a surface, and OLS shows the same trade-off to a lesser extent. LatentAM, whose zero-degree spherical harmonics are similar to our RGB colors, has the best depth on Replica but the worst images. OREN-X renders along its SDF surface, balancing the two, with the second-best depth and better images than LatentAM on Replica, and clearly the best depth on the real TUM sequences.

E. Open-Vocabulary Understanding

In Table III, one of the two OREN-X backbones is best in 46 of the 48 Replica scene-metric entries. On TUM, OREN-X with TIPSv2-L remains best on all 3D query metrics and improves mIoU and mAcc by nearly 50%, while LatentAM and OLS lead in 2D since they are optimized specifically for 2D. OLS reconstructs 2D features best in every Replica scene but trails in text queries. OREN-X with CLIP mostly has the lowest fidelity but is best in most mIoU and mAcc entries. So, VL fidelity does not determine accuracy.

F. Ablation Studies

Unless stated otherwise, ablations use Replica room0.

Efficiency. Table IV reports throughput and peak GPU memory as modalities are added to OREN-X. Each added

TABLE II: **Photometric** quality on Replica and TUM (mean over each dataset). Best in **bold** and shaded green, second yellow.

Method	PSNR↑		SSIM↑		LPIPS↓		Depth L1 (mm)↓	
	Replica	TUM	Replica	TUM	Replica	TUM	Replica	TUM
LangSplatV2 [30]	36.00	19.74	0.96	0.78	0.0683	0.2281	38.50	77.72
OLS [35]	32.38	15.61	0.91	0.62	0.2359	0.4377	17.52	36.21
LatentAM [11]	23.77	12.84	0.76	0.49	0.3789	0.6057	6.29	54.98
OREN-X (ours)	28.01	14.21	0.88	0.56	0.2749	0.5038	10.02	17.12

modality lowers throughput moderately and the VL modality costs the most, yet OREN-X runs all four modalities in real time within about 1 GiB of GPU memory, and with the SDF alone is faster than nvblox and uses less memory. Among the VL methods, LatentAM reaches a similar throughput but needs about eight times more memory, while OnlineLangSplat and LangSplatV2 are one to two orders of magnitude slower.

Per-vertex bias correction. Compared with vanilla Adam, per-vertex bias correction consistently reduces the near-surface SDF, gradient and depth errors, while SSIM is unaffected (Fig. 6), since the correction matters for rarely observed vertices near the surface, and updating moments only for active vertices lowers peak memory by about a third and reduces loss jitter (Fig. 7).

Cross-modality synergy and depth noise. Table V and Fig. 8 add modalities under increasing depth noise. Radiance reduces the SDF error most on clean depth, as photometric gradients pull the zero level set onto the surface but its benefit shrinks as noise grows. Occupancy helps only under strong noise, where confident logits correct the SDF sign, and all modalities are best on clean depth and the strongest noise.

SDF-induced surface location. Replacing the segment midpoint with the SDF zero crossing (Table VI) leaves PSNR and SSIM unchanged but reduces LPIPS by 7%, indicating better perceptual quality.

Radiance representation. Spherical harmonics (Table VII) improve quality only marginally on the largely Lambertian Replica scenes, using $2.5\times$ memory, so we keep RGB.

Train/eval rasterizer. In Table VIII, training with rays gives better quality with either evaluation backend and faster steps at more memory, while the tile-based rasterizer renders two orders of magnitude faster at a small quality cost, so the same octree serves both quality and speed.

Online dictionary. In Table IX, the online dictionary gives the best mAP at about a quarter of the memory of full features, since discarding low-variance directions removes noise from the cosine score. An implicit MLP stores the VL field in even less memory but loses substantial accuracy, and its features must be decoded before querying. An offline PCA dictionary degrades when centered, because the codes omit the feature mean, which dominates VL features (Fig. 4a). Accuracy saturates at $G_{\max}=256$, as the energy criterion stops adding atoms before reaching capacity, so a larger $G_{\max}$ only widens the stored coefficient vectors, which is why memory keeps growing. Sparse top- K codes with fewer atoms keep most accuracy, using less memory.

VL backbone. Table X compares four VL backbones on Replica. OREN-X reconstructs TIPSv2-L features best and DINOv3.txt features worst, consistent with Fig. 4a, where TIPSv2-L features concentrate most tightly around their

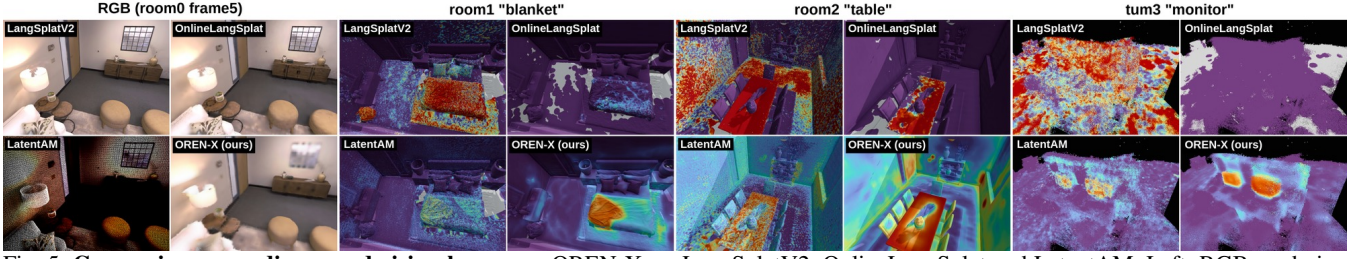


Fig. 5: **Comparison on radiance and vision-language.** OREN-X vs. LangSplatV2, OnlineLangSplat and LatentAM. Left: RGB rendering of Replica room0. Remaining columns: open-vocabulary query of room1 (“blanket”), room2 (“table”) and TUM 3 (“monitor”).

TABLE III: **Open-vocabulary** understanding on Replica and TUM (mean). Best per column shaded green and bold, second yellow.

Method	Metric $\uparrow$	Room0		Room1		Room2		Office0		Office1		Office2		Office3		Office4		TUM (mean)	
		3D	2D	3D	2D	3D	2D	3D	2D	3D	2D	3D	2D	3D	2D	3D	2D	3D	2D
LangSplatV2 (CLIP)	Cos-Sim	0.858	0.918	0.882	0.924	0.865	0.923	0.885	0.917	0.893	0.929	0.864	0.914	0.852	0.904	0.868	0.921	0.783	0.903
	mIoU	19.10	28.00	18.40	26.80	18.30	25.00	13.30	23.60	10.00	12.30	21.60	34.70	15.30	26.70	23.40	29.80	9.57	42.10
	mAcc	29.80	36.00	32.80	37.70	31.90	40.30	22.60	35.90	24.10	27.60	30.70	42.90	26.00	35.90	41.80	47.20	28.30	61.77
	mAP	24.10	32.70	26.80	36.00	25.30	31.40	25.70	31.20	19.20	27.70	26.60	32.70	21.80	29.50	31.70	42.00	16.83	50.13
OLS (CLIP)	Cos-Sim	0.738	0.945	0.854	0.952	0.768	0.951	0.830	0.939	0.854	0.938	0.765	0.934	0.790	0.939	0.814	0.942	0.864	0.901
	mIoU	20.30	30.00	21.00	33.10	24.40	30.90	13.70	24.60	9.90	9.80	20.70	28.80	15.30	23.40	19.60	21.20	22.23	43.40
	mAcc	30.50	38.40	35.20	45.40	36.50	40.70	26.10	37.80	18.50	18.40	28.60	34.50	22.50	29.80	29.90	31.10	34.27	53.10
	mAP	37.40	40.10	39.00	41.70	41.00	40.10	24.10	26.30	18.40	20.60	29.90	34.30	28.60	30.80	38.90	35.50	32.77	55.30
LatentAM (TIPSv2-L)	Cos-Sim	0.799	0.819	0.793	0.821	0.791	0.811	0.797	0.813	0.810	0.826	0.804	0.817	0.791	0.817	0.804	0.821	0.733	0.803
	mIoU	20.00	28.60	18.20	31.20	18.40	21.80	19.30	23.70	14.90	21.80	16.20	26.20	14.70	22.90	22.50	23.40	16.93	50.77
	mAcc	34.60	44.80	36.70	52.80	34.20	41.70	32.90	43.50	28.50	33.20	30.40	37.90	26.60	36.30	38.90	47.70	31.60	67.70
	mAP	31.80	32.40	28.40	35.90	29.80	34.20	29.10	31.60	26.10	27.10	25.20	30.70	23.70	27.10	35.50	34.00	23.13	51.33
OREN-X (TIPSv2-L)	Cos-Sim	0.898	0.884	0.880	0.886	0.888	0.883	0.867	0.876	0.894	0.884	0.872	0.872	0.876	0.885	0.894	0.875	0.850	0.846
	mIoU	33.09	37.34	28.27	34.81	27.07	27.51	30.13	28.82	23.41	25.00	32.43	34.90	26.38	27.99	27.24	28.21	33.21	48.86
	mAcc	50.60	53.98	54.83	61.22	47.92	47.81	48.60	47.12	37.94	38.56	56.70	54.14	43.92	44.00	50.17	53.33	50.42	61.17
	mAP	49.02	39.81	50.83	47.48	44.91	41.13	38.17	33.75	34.90	31.91	46.50	38.16	35.20	31.52	39.62	36.60	34.18	45.85
OREN-X (CLIP)	Cos-Sim	0.790	0.776	0.776	0.776	0.773	0.766	0.753	0.756	0.763	0.757	0.765	0.762	0.761	0.769	0.766	0.753	0.761	0.756
	mIoU	27.80	30.77	28.74	34.04	31.09	32.69	28.13	26.60	25.00	28.43	39.37	40.66	29.35	29.03	38.93	42.22	19.42	34.21
	mAcc	50.66	51.79	53.19	61.24	53.05	54.81	46.92	47.85	44.99	44.11	61.72	57.89	49.70	46.34	62.48	66.72	39.70	47.70
	mAP	42.85	36.67	44.76	48.76	48.35	44.05	39.38	32.87	28.40	29.61	46.25	35.07	35.28	34.18	43.20	40.36	34.12	42.85

TABLE IV: **Efficiency on Replica room0.** Throughput and peak GPU memory of OREN-X as modalities are added, and of the baselines.

Metric	OREN-X					Baselines			
	SDF	SDF+OCC	SDF+Rad	SDF+OCC+Rad	SDF+OCC+Rad+VL	LatentAM	OnlineLangSplat	LangSplatV2	nvblox
FPS $\uparrow$	80.06	71.72	60.89	54.07	32.37	34.11	1.79	0.42	73.66
GPU Peak (GiB) $\downarrow$	0.28	0.34	0.78	0.87	1.01	8.66	11.32	13.18	1.00

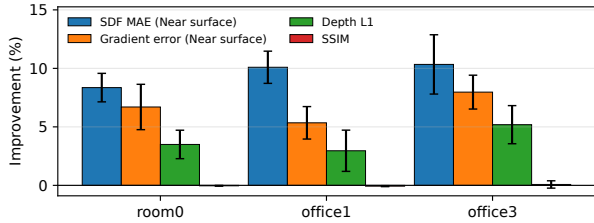


Fig. 6: **Per-vertex bias correction vs. vanilla Adam.** Relative improvement from per-vertex bias correction on near-surface SDF MAE, near-surface gradient error, mean depth L1 and SSIM. Error bars are calculated over five seeds.

TABLE V: **Depth-noise robustness.** Replica room0. Near-surface SDF MAE (mm) across injected depth-noise levels, for Gaussian noise and axial noise. Best per column in **bold**, shaded green.

Variant	No Noise	Gaussian σ			Axial σ		
		0.01	0.02	0.05	0.0011	0.0023	0.0057
SDF	15.59	13.67	13.55	20.07	14.66	13.88	24.32
SDF+OCC	15.83	13.78	13.38	19.70	14.90	14.06	23.91
SDF+Rad	12.06	12.04	12.95	18.66	11.87	12.97	24.19
SDF+OCC+Rad	11.40	12.94	13.47	19.23	12.20	13.47	23.81

mean and DINOv3.txt features spread the most. CLIP gives the best mAcc and Talk2DINOv3-ViT-L the best mIoU and mAP. Reconstruction fidelity again does not determine query accuracy, and the same dictionary hyperparameters work well across all four backbones without tuning.

VI. CONCLUSION

We presented OREN-X, an online multi-modal mapping framework that maintains SDF, occupancy, radiance and

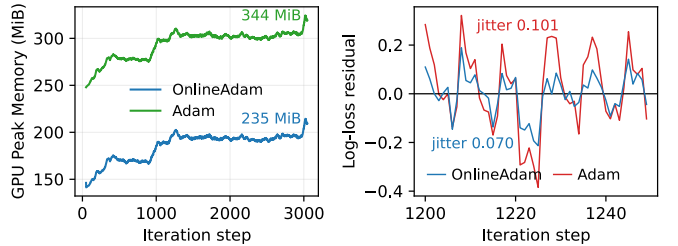


Fig. 7: **Per-vertex bias correction vs. vanilla Adam: memory and stability.** Left: GPU peak memory over training with the SDF, OCC and radiance modalities on Replica room0. Right: detrended loss residual over a window. The legend reports each optimizer’s jitter (the standard deviation of this residual over the whole run).

TABLE VI: **SDF-induced surface location.** Replica room0. Rendering with the sample placed at the SDF zero crossing vs. the segment midpoint. Best per column in **bold**, shaded green.

Variant	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
SDF zero crossing	28.35	0.87	0.2978	64.84
Segment midpoint	28.36	0.88	0.3209	66.28

VL fields in a single octree with a common storage representation, shared indexing and joint training. Joint training improves SDF accuracy, shared indexing enables real-time rendering and both 2D and 3D open-vocabulary queries, and an online dictionary compresses the VL field while improving query accuracy. On Replica, OREN-X maps in real time and outperforms prior methods in SDF accuracy and open-vocabulary segmentation. On the real-world TUM RGB-D sequences [40], it achieves the lowest depth error

TABLE VIII: **Train/eval rasterizer.** Replica `room0`. Ray- vs. tile-based rasterizer for training and evaluation. Train FPS and Mem. (peak GPU memory) depend only on the training rasterizer, and Eval FPS is measured at 1200×680 . Best per column in **bold**.

Train/Eval	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Train FPS $\uparrow$	Eval FPS $\uparrow$	Mem.(MiB) $\downarrow$
ray/ray	28.32	0.8742	0.2981	31.80	12.0	341.1
ray/tile	27.68	0.8703	0.3036		1513.8	
tile/ray	26.77	0.8751	0.3154	29.25	12.0	
tile/tile	27.00	0.8751	0.3138		1456.7	262.5

TABLE IX: **Online dictionary.** Replica `room0`. VL field stored as full per-vertex features, as implicit features decoded by an MLP, as codes over an offline PCA dictionary (with/without centering), and as codes from our online dictionary, with dense coefficients for varying $G_{\max}$ or sparse top- K coefficients at $G_{\max}=256$. $|D|$ is the number of atoms the growth criterion activated. It equals $G_{\max}$ for the fixed-size offline dictionaries. Dense rows pad each coefficient vector to $G_{\max}$. Top- K rows store K coefficients and their indices. Mem. is the storage of the VL field (MB). Best per column in **bold**, shaded green.

Variant	$G_{\max}$	$ D $	Cosine $\uparrow$	mAP $\uparrow$	Mem. $\downarrow$	FPS $\uparrow$
Full features	–	–	0.907	45.87%	256.2	31.00
Implicit MLP	–	–	0.902	41.59%	9.5	31.06
Offline (uncentered)	256	256	0.905	46.66%	69.3	32.37
Offline (centered)	256	256	0.736	44.54%	69.3	32.37
Online, dense coeff.	64	64	0.845	40.96%	20.5	–
Online, dense coeff.	128	128	0.884	45.38%	36.8	–
Online, dense coeff.	256	228	0.898	49.02%	69.3	32.37
Online, dense coeff.	512	228	0.898	49.02%	134.3	–
Online, dense coeff.	1024	228	0.898	49.02%	264.3	–
Online, top- $K=4$	256	228	0.795	32.74%	6.8	29.99
Online, top- $K=48$	256	228	0.878	45.38%	23.3	29.99

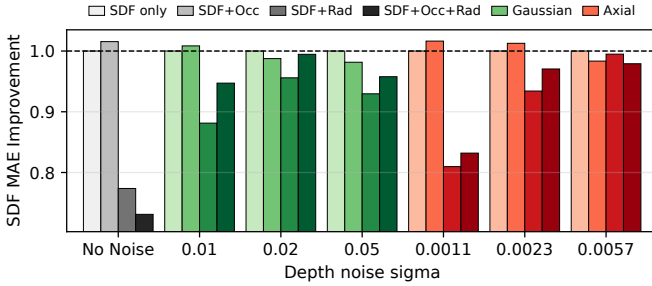


Fig. 8: **Depth-noise robustness.** Replica `room0`. Near-surface SDF MAE of each modality combination, normalized by the SDF-only variant at the same noise level (lower is better, the dashed line is SDF-only). Two noise models are used: Gaussian noise $\mathcal{N}(0, \sigma^2)$ and axial noise $\mathcal{N}(0, (\sigma z)^2)$, where z is the depth value.

TABLE VII: **RGB vs. spherical-harmonic (SH) color.** Replica `room0`. Mem. is peak GPU memory per training step, and FPS is training throughput. Best per column in **bold**, shaded green.

Repr.	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Mem.(MiB) $\downarrow$	FPS $\uparrow$
RGB	28.32	0.8742	0.2981	242.8	32.75
SH1	28.30	0.8737	0.2953	313.8	32.07
SH2	28.40	0.8743	0.2947	433.5	31.92
SH3	28.46	0.8746	0.2942	608.4	31.45

and the best 3D open-vocabulary segmentation.

REFERENCES

- [1] H. Oleynikova, Z. Taylor, M. Fehr *et al.*, “Voxblox: Incremental 3D Euclidean signed distance fields for on-board MAV planning,” in *IROS*, 2017.
- [2] A. Millane, H. Oleynikova, E. Wirbel *et al.*, “nvblox: GPU-accelerated incremental signed distance field mapping,” in *ICRA*, 2024.
- [3] J. Ortiz, A. Clegg, J. Dong *et al.*, “iSDF: Real-time neural signed distance fields for robot perception,” in *RSS*, 2022.
- [4] V. Vasilopoulos, S. Garg, J. Huh *et al.*, “HIO-SDF: Hierarchical incremental online signed distance fields,” in *ICRA*, 2024.
- [5] Y. Pan, X. Zhong, L. Wiesmann *et al.*, “PIN-SLAM: LiDAR SLAM using a point-based implicit neural representation for achieving global map consistency,” *IEEE T-RO*, 2024.
- [6] Z. Dai, Q. Qian, T. Fan, and N. Atanasov, “OREN: Octree Residual Network for Real-Time Euclidean Signed Distance Mapping,” in *IROS*, 2026.
- [7] C. Jiang, Y. Luo, B. Zhou, and S. Shen, “H3-Mapping: Quasi-heterogeneous feature grids for real-time dense mapping using hierarchical hybrid representation,” *IEEE RA-L*, 2024.
- [8] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM TOG*, 2023.
- [9] J. Kerr, C. M. Kim, K. Goldberg *et al.*, “LERF: Language embedded radiance fields,” in *ICCV*, 2023.
- [10] M. Qin, W. Li, J. Zhou *et al.*, “LangSplat: 3D language Gaussian splatting,” in *CVPR*, 2024.
- [11] J. Lee and Y. Tian, “LatentAM: Real-time, large-scale latent Gaussian attention mapping via online dictionary learning,” *IEEE RA-L*, 2026.
- [12] B. Curless and M. Levoy, “A volumetric method for building complex models from range images,” in *SIGGRAPH*, 1996.
- [13] R. A. Newcombe, S. Izadi, O. Hilliges *et al.*, “KinectFusion: Real-time dense surface mapping and tracking,” in *ISMAR*, 2011.
- [14] A. Hornung, K. M. Wurm, M. Bennewitz *et al.*, “OctoMap: An efficient probabilistic 3D mapping framework based on octrees,” *Autonomous Robots*, 2013.
- [15] L. Mescheder, M. Oechsle, M. Niemeyer *et al.*, “Occupancy networks: Learning 3D reconstruction in function space,” in *CVPR*, 2019.
- [16] T. Takikawa, J. Litalien, K. Yin *et al.*, “Neural geometric level of detail: Real-time rendering with implicit 3D shapes,” in *CVPR*, 2021.
- [17] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM TOG*, 2022.
- [18] C. Sun, J. Choe, C. Loop *et al.*, “Sparse voxels rasterization: Real-time high-fidelity radiance field rendering,” in *CVPR*, 2025.
- [19] B. Mildenhall, P. P. Srinivasan, M. Tancik *et al.*, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *ECCV*, 2020.
- [20] A. Yu, R. Li, M. Tancik *et al.*, “PlenOctrees for real-time rendering of neural radiance fields,” in *ICCV*, 2021.
- [21] P. Wang, L. Liu, Y. Liu *et al.*, “NeuS: Learning neural implicit surfaces by volume rendering for multi-view reconstruction,” in *NeurIPS*, 2021.
- [22] B. Huang, Z. Yu, A. Chen *et al.*, “2D Gaussian splatting for geometrically accurate radiance fields,” in *SIGGRAPH*, 2024.
- [23] A. Asgharivaskasi and N. Atanasov, “Semantic OcTree mapping and Shannon mutual information computation for robot exploration,” *IEEE T-RO*, 2023.
- [24] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A real-time spatial perception system for 3D scene graph construction and optimization,” in *RSS*, 2022.
- [25] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *ICML*, 2021.
- [26] O. Siméoni, H. V. Vo, M. Seitzer *et al.*, “DINOv3,” arXiv preprint arXiv:2508.10104, 2025.
- [27] C. Jose, T. Moutakanni, D. Kang *et al.*, “DINOv2 meets text: A unified framework for image- and pixel-level vision-language alignment,” in *CVPR*, 2025.
- [28] L. Barsellotti, L. Bianchi, N. Messina *et al.*, “Talking to DINO: Bridging self-supervised vision backbones with language for open-vocabulary segmentation,” in *ICCV*, 2025.
- [29] B. Cao, K. Chen, K.-K. Maninis *et al.*, “TIPSv2: Advancing vision-language pretraining with enhanced patch-text alignment,” in *CVPR*, 2026.
- [30] W. Li, Y. Zhao, M. Qin *et al.*, “LangSplatV2: High-dimensional 3D language Gaussian splatting with 450+ FPS,” in *NeurIPS*, 2025.
- [31] M. Wang, J.-C. Shi, S.-H. Guan, and H.-B. Duan, “Language embedded 3D Gaussians for open-vocabulary scene querying,” *IEEE TPAMI*, 2026.
- [32] S. Peng, K. Genova, C. Jiang *et al.*, “OpenScene: 3D scene understanding with open vocabularies,” in *CVPR*, 2023.
- [33] A. Werby, C. Huang, M. Büchner *et al.*, “Hierarchical open-vocabulary 3D scene graphs for language-grounded robot navigation,” in *RSS*, 2024.
- [34] Z. Wang, Y. Su, C. Li *et al.*, “Open-vocabulary octree-graph for 3D scene understanding,” in *ICCV*, 2025.
- [35] S. Katragadda, C.-Y. Wu, Y. Guo *et al.*, “Online language splatting,” in *ICCV*, 2025.
- [36] J. Mairal, F. Bach, J. Ponce, and G. Sapiro, “Online dictionary learning for sparse coding,” in *ICML*, 2009.

TABLE X: **VL backbone comparison.** Replica (8-scene mean). VL reconstruction and open-vocabulary query quality of OREN-X with four VL backbones, each cell 3D/2D. Best per 3D or 2D half in **bold**, shaded green, second yellow.

Backbone	Cos-Sim (3D/2D)↑	mIoU (3D/2D)↑	mAcc (3D/2D)↑	mAP (3D/2D)↑
CLIP ViT-B/16	0.768/ 0.764	31.05 / 33.06	52.84 / 53.84	41.06/ 37.69
DINOv3.txt	0.684/0.694	27.96/30.59	43.72/45.66	39.40/35.25
Talk2DINOv3-ViT-L	0.771 /0.748	33.50 / 35.43	46.87/48.01	42.42 / 40.46
TIPsv2-L	0.884 / 0.881	28.50/30.57	48.83 / 50.02	42.40 /37.54

[37] J. Revelles, C. Ureña, and M. Lastra, “An efficient parametric algo-

rithm for octree traversal,” in *WSCG*, 2000.

- [38] H. Cardot and D. Degras, “Online principal component analysis in high dimension: Which algorithm to choose?” *International Statistical Review*, 2018.
- [39] J. Straub, T. Whelan, L. Ma *et al.*, “The Replica dataset: A digital replica of indoor spaces,” arXiv preprint arXiv:1906.05797, 2019.
- [40] J. Sturm, N. Engelhard, F. Endres *et al.*, “A benchmark for the evaluation of RGB-D SLAM systems,” in *IROS*, 2012.
- [41] N. Carion, L. Gustafson, Y.-T. Hu *et al.*, “SAM 3: Segment anything with concepts,” in *ICLR*, 2026.